

LAB6

Bu haftaki lab etkinliğimizde elimizdeki lopy4'ler içinde TinyML çalıştırmayı denedik. Elimizde ise şöyle bi repo vardı: https://github.com/tensorflow/tflite-micro/tree/main/tensorflow/lite/micro/examples/hello_world . Aşağıda adım adım ChatGPT ile yaptığımız konuşmaları ve aldığımız hataları veriyorum.

- Yapay zekanın verdiği genel akış şöyleydi ve biz bunu yapmaya başladık:

Senin elindeki repo:

👉 TensorFlow Lite Micro examples / hello_world

Bu örnek, C++'ta yazılmış ve **ARM Cortex-M tabanlı** mikrodenetleyicilerde çalışan bir “sinyüs dalgası tahmini” modeli (tiny ML) örneğidir.

Ancak LoPy4:

- ESP32 tabanlı (Xtensa LX6 işlemci),
- MicroPython firmware'i ile çalışıyor,
- Yani doğrudan C++ kodu değil, .py veya C olarak gömülmüş modüller çalıştırabiliyor.

Bu yüzden doğrudan make ile bu örneği çalıştıramazsın — **önce portlama** gerekiyor.

🔧 2 İki temel yaklaşım

TensorFlow Lite Micro'yu LoPy4'te çalıştırmak için 2 olasılık var:

Yaklaşım	Açıklama
A. MicroPython içine gömülü C modülü olarak derlemek	TFLM kütüphanesini C++ modül olarak LoPy firmware'ine entegre edip, MicroPython üzerinden çağırmak. Bu en düşük seviye ama en hızlı yöntem.
B. Modeli Python tarafında yorumlamak (edge impulse / tflite-interpreter)	LoPy'nin gücü sınırlı, bu yüzden TinyML modelini Raspberry Pi veya PC'de çalıştırıp sadece sensör verisini LoRaWAN üzerinden göndermek. Bu “uç cihazda inference” değil, ama pratik bir yol.

Senin “TFLite Micro doğrudan LoPy4 içinde çalışsın” isteğine göre ben **Yöntem A'yı** anlatayım 👉

3 Gerekli araçlar

LoPy4 için firmware derlemek ve içine TFLM gömmek için şunlar gerekli:

Bilgisayar tarafında:

- **Linux / macOS** ortamı (Windows da olur ama WSL tavsiye edilir)
- git, make, cmake, python3, pip
- **Pycom firmware tools** (LoPy4 firmware derleme araçları)
👉 <https://github.com/pycom/pycom-micropython-sigfox>
(Bu repo MicroPython'un Pycom fork'udur)

Kurulum:

```
git clone https://github.com/pycom/pycom-micropython-sigfox.git
```

```
cd pycom-micropython-sigfox
```

```
git submodule update --init --recursive
```

4 TensorFlow Lite Micro'yu Pycom'a entegre et

Bu adım "port" adıımıdır:

1. TFLite Micro repo'sunu al:
2. `git clone https://github.com/tensorflow/tflite-micro.git`
3. tensorflow/lite/micro içindeki dosyaları Pycom firmware içindeki uygun bir yere kopyala:
örn:
4. `pycom-micropython-sigfox/esp32/tflm/`
5. Pycom build sistemine (örneğin `esp32/application.mk`) aşağıdaki gibi dahil et:
6. `# TensorFlow Lite Micro`
7. `CFLAGS += -I$(TOP)/esp32/tflm/tensorflow/lite/micro`
8. `CXXFLAGS += -I$(TOP)/esp32/tflm/tensorflow/lite/micro`
9. `SRC_CXX += $(wildcard $(TOP)/esp32/tflm/tensorflow/lite/micro/*.cc)`
10. hello_world örneğini bir C++ modül haline getir:
`tensorflow/lite/micro/examples/hello_world/main_functions.cc` ve `constants.cc` gibi dosyaları kullanarak bir `tflm_hello_module.c` oluştur.
Bu modül MicroPython içinde çağrılabilir hale gelir.

5 Python'dan erişim

Yeni modülün içinde bir fonksiyon tanımlarsın:

```
extern "C" {  
  
    mp_obj_t tflm_hello_run(void) {  
  
        setup();  
  
        loop();  
  
        return mp_const_none;  
  
    }  
  
    MP_DEFINE_CONST_FUN_OBJ_0(tflm_hello_run_obj, tflm_hello_run);  
  
}
```

Böylece MicroPython tarafında:

```
import tflm_hello
```

```
tflm_hello.run()
```

yazarak inference çalıştırabilirsin 🎉

6 Derleme ve yükleme

```
cd pycom-micropython-sigfox
```

```
make BOARD=LOPY4
```

Sonra çıkan firmware (.bin) dosyasını:

- LoPy4'e **Pycom Firmware Update Tool** veya esptool.py ile flashla:
 - `esptool.py --port /dev/ttyUSB0 write_flash -z 0x1000 build/LOPY4/firmware-combined.bin`
-

7 Test

VSCoDe + Pymakr ile LoPy4'e bağlan,
REPL üzerinden:

```
import tflm_hello
```

```
tflm_hello.run()
```

komutu verdiğinde sinüs dalgası tahmin çıktısını (örnek olarak “y: 0.33”) görmelisin.

- Fakat şöyle bi hata aldık kurulumlarda:
(base) bilmuh@lab3pc22:~/lopy4\$ ls main2.py main.py pycom-micropython-sigfox pymakr.conf (base)
bilmuh@lab3pc22:~/lopy4\$ cd pycom-micropython-sigfox (base)
bilmuh@lab3pc22:~/lopy4/pycom-micropython-sigfox\$ git submodule update --init --recursive
fatal: .git/index: indeks dosyası beklenenden daha küçük
- Kopyalamada sorun yaşamışız şu adımlarla çözdük:

Git deposu bozulduğunda (özellikle yarım kopyalama veya disk yazımı kesilirse) ortaya çıkar.

Yani pycom-micropython-sigfox klasörünün .git/index dosyası hasar görmüş.

Bunu **tamir etmek** için birkaç güvenli adım var 🙌

1 Git index'i yeniden oluştur

Terminalde:

```
cd ~/lopy4/pycom-micropython-sigfox
```

```
rm -f .git/index
```

```
git reset
```

Bu, bozuk index dosyasını silip HEAD'deki sürümle yeniden oluşturur.

Eğer çıktı düzgün gelirse, sonra tekrar:

```
git submodule update --init --recursive
```

2 Eğer yukarıdaki işe yaramazsa

Depo ciddi bozulmuş olabilir. O zaman en temiz çözüm:

Eski klasörü yedekle (isteğe bağlı):

```
cd ~/lopy4
```

```
mv pycom-micropython-sigfox pycom-micropython-sigfox.bak
```

Yeniden klonla:

```
git clone https://github.com/pycom/pycom-micropython-sigfox.git
```

```
cd pycom-micropython-sigfox
```

```
git submodule update --init --recursive
```

Bu işlem repo'yu tertemiz çeker.

Daha sonra senin LoPy4'e özgü değişiklikleri (örneğin main.py, modül eklemeleri vb.) eski klasörden kopyalayabilirsin.

- Genel akıştan devam ederken bu hatayı aldık:
(base) bilmuh@lab3pc22:~/lopy4\$ rsync -av --progress tflite-micro/tensorflow/lite/micro/ pycom-micropython-sigfox/esp32/tflm/ Unknown --info item: "flist" rsync error: syntax or usage error (code 1) at options.c(467) [client=3.2.7]

Sorun şu:

Ubuntu'nun bazı sürümlerinde (veya Conda ortamında yüklü rsync sürümlerinde) --progress parametresiyle birlikte kullanılan iç mekanizma (--info=flist2) artık farklı çalışıyor.

Bizim komutta --progress kendi içinde --info=flist seçeneğini tetikliyor, ama senin rsync sürümün (3.2.7) bunu desteklemiyor.

Yani hatanın özü:

Unknown --info item: "flist"

✅ 1 Kolay çözüm: --progress'i kaldır

Aynı komutu **--progress olmadan** çalıştır:

```
rsync -av tflite-micro/tensorflow/lite/micro/ pycom-micropython-sigfox/esp32/tflm/
```

Bu şekilde dosyalar kopyalanır, sadece ilerleme çubuğu gösterilmez.

✅ 2 Alternatif: cp -r kullan (daha basit ama biraz riskli)

```
cp -r tflite-micro/tensorflow/lite/micro/* pycom-micropython-sigfox/esp32/tflm/
```

Bu da tüm alt dizinleriyle birlikte kopyalar.

(Orijinal dosyalar bozulmaz, sadece kopyalanır.)

✅ 3 Daha temiz ve hızlı seçenek: sembolik link

Bu, yer kaplamaz ve kopyalamaya gerek yok:

```
cd pycom-micropython-sigfox/esp32
```

```
ln -s ../../tflite-micro/tensorflow/lite/micro tflm
```

Sonra ls -l ile kontrol et:

```
ls -l tflm
```

şuna benzer bir çıktı görmelisin:

```
tflm -> ../../tflite-micro/tensorflow/lite/micro
```

- Bu yanıtın sonra verilenleri yapmadık ama conda deactivate yapıp tekrar denedik aynı hatayı aldık.

Conda ortamından çıkman çok iyi olmuş 🙌

(Çünkü conda bazen sistem rsync yerine kendi eski sürümünü çağırıyor.)

Ama hâlâ aynı hatayı veriyorsa, sorun **--progress parametresinden** kaynaklı.

Bu yüzden şimdi **en sade ve garantili yöntemle** ilerleyelim 🙌

✅ 1 En garantili çözüm: --progress olmadan kopyala

Terminalde çalıştır:

```
rsync -av tflite-micro/tensorflow/lite/micro/ pycom-micropython-sigfox/esp32/tflm/
```

Bu komut:

- tflite-micro/tensorflow/lite/micro içindeki tüm dosyaları
- pycom-micropython-sigfox/esp32/tflm klasörüne **alt izinlerle birlikte** kopyalar. Hiçbir şey silmez, sadece ekleme yapar.

✅ 2 Alternatif: Sembolik link (yer kaplamaz)

İstersen (benim önerdiğim daha temiz yöntem):

```
cd ~/lopy4/pycom-micropython-sigfox/esp32
```

```
ln -s ../../tflite-micro/tensorflow/lite/micro tflm
```

Doğru bağlandığını kontrol etmek için:

```
ls -l tflm
```

çıktı şöyle olmalı:

tflm -> ../../tflite-micro/tensorflow/lite/micro

Bu durumda artık dosya kopyalamana gerek yok,
Pycom build sistemi bu yolu kullanabilir.

- –progress olmadan kopyaladık ve bu adımlarla sorun çözüldü.
- Kopyalama kısmı tamam olduğuna göre, şimdi **TFLite Micro'nun "hello_world" örneğini**, Pycom firmware içinde **çağrılabilir bir C modülü (extension module)** haline getireceğiz, adımıyla devam ederken aldığımız diğer bi hata:

```
bilmuh@lab3pc22:~/lopy4$ cp tflite-  
micro/tensorflow/lite/micro/examples/hello_world/*.h pycom-micropython-  
sigfox/esp32/tflm/ cp: 'tflite-  
micro/tensorflow/lite/micro/examples/hello_world/*.h' durumlanamadı: Böyle bir  
dosya ya da dizin yok  
bilmuh@lab3pc22:~/lopy4$ ls main2.py main.py pycom-micropython-sigfox  
pymkr.conf tflite-micro  
bilmuh@lab3pc22:~/lopy4$ cd pycom-micropython-sigfox/  
bilmuh@lab3pc22:~/lopy4/pycom-micropython-sigfox$ cd ..  
bilmuh@lab3pc22:~/lopy4$ cp tflite-  
micro/tensorflow/lite/micro/examples/hello_world/*.h pycom-micropython-  
sigfox/esp32/tflm/ cp: 'tflite-  
micro/tensorflow/lite/micro/examples/hello_world/*.h' durumlanamadı: Böyle bir  
dosya ya da dizin yok
```

hello_world klasöründe gerçekten **.h uzantılı dosya olmayabilir** (bazı TFLM versiyonlarında sadece .cc dosyaları var).

Yani hata “dosya yok” diyor çünkü gerçekten yok.

Bunu kesinleştirelim.

1 Önce dizine bakalım

Lütfen şu komutu çalıştır:

```
ls tflite-micro/tensorflow/lite/micro/examples/hello_world/
```

Bu çıktıyı bana at — ama muhtemelen şunlar gibi bir şey göreceksin:

```
main_functions.cc
```

```
constants.cc
```

```
constants.h
```

```
model_data.cc
```

```
model_data.h
```

ya da bazen sadece .cc dosyaları olur (örneğin constants.h ve model_data.h olmadan).

✅ 2 Eğer .h dosyaları varsa

O zaman komutun doğru, sadece * shell genişletmesi çalışmamış olabilir.

Bu durumda tam yolu kullan:

```
cp tflite-micro/tensorflow/lite/micro/examples/hello_world/constants.h pycom-micropython-sigfox/esp32/tflm/
```

```
cp tflite-micro/tensorflow/lite/micro/examples/hello_world/model_data.h pycom-micropython-sigfox/esp32/tflm/
```

❌ 3 Eğer .h dosyaları yoksa

Hiç sorun değil — TFLM’in bazı sürümlerinde bu dosyalar *.cc içine gömülü (örneğin model verisi const unsigned char g_model[] olarak .cc içinde tanımlanıyor).

Yani .h olmadan da devam edebiliriz.

Yani özetle:

- .h dosyaları varsa → tek tek kopyalayalım.
- .h dosyaları yoksa → direkt main_functions.cc ve constants.cc dosyalarıyla devam edeceğiz.

Bana şu çıktıyı atabilir misin? 🙋

```
ls tflite-micro/tensorflow/lite/micro/examples/hello_world/
```

- Biraz daha konuşup repoyu kaldırıp tekrar klonladıktan sonra bizde .h’lı dosyaların olmadığını anladık. Devamında ise şu adımlarla tekrar denedik:

Amacın sadece **LoPy4 üzerinde TensorFlow Lite Micro “hello world”** örneğini çalıştırmak, modelin detaylarına girmeye gerek yok diyorsun.

Ama şöyle bir gerçek var:

Senin şu anda sahip olduğun hello_world klasörü **Python tarafındaki eğitim örneği**, yani .py dosyaları TensorFlow (masaüstü) üzerinde eğitimi simüle ediyor.

Bunu **LoPy4 üzerinde** çalıştıramayız, çünkü MicroPython içinde ne TensorFlow var ne de NumPy.

💡 O zaman mantıklı rota şu:

Senin elindeki cihaz (LoPy4) yalnızca **C tabanlı TFLite Micro** derlemelerini çalıştırabiliyor.

Yani bizim, .cc tabanlı hello_world örneğini içeren sürüme ihtiyacımız var. Seninki TensorFlow “ana” deposundan geliyor, TFLite Micro “standalone” sürüm değil.

✅ Şunu yapalım (emin olalım):

Eski klonun içinde bir karışıklık var gibi, o yüzden bu komutlarla **tam temiz ve doğru** repoyu alalım:

```
cd ~/lopy4
```

```
rm -rf tflite-micro
```

```
git clone https://github.com/tensorflow/tflite-micro.git
```

```
cd tflite-micro
```

```
git submodule update --init --recursive
```

Sonra kontrol et:

```
ls tensorflow/lite/micro/examples/hello_world/
```

Eğer bu komuttan sonra yine .cc dosyaları gelmiyorsa, alternatif yol olarak şunu yapacağız 🙋

🔧 Alternatif Plan (LoPy4’de çalışabilir basit demo)

Eğer bu dosyalar yine gelmezse, biz:

1. TFLite Micro’yu minimal şekilde kendimiz entegre edeceğiz,
2. “Hello World” yerine **basit bir sinüs tahmin demo’su** ($\sin(x)$ hesaplayan NN modeli) çalıştıracacağız.

Bu örnek zaten “hello_world” modelinin yaptığı şeyin aynısı.

Sadece senin repona bağımsız, doğrudan LoPy4 içinde derlenebilir olacak.

```
bilmuh@lab3pc22:~/lopy4/tflite-micro$ ls tensorflow/lite/micro/examples/hello_world/  
BUILD evaluate_test.py images models README.md evaluate.py hello_world_test.cc  
Makefile.inc quantization train.py
```

Tamam 🙋 çok güzel bir yere geldik — şu anda gördüğün klasör (yani evaluate.py, train.py, models/ klasörü olan) aslında **TensorFlow Lite Micro’nun eğitim (Python) kısmı**.

Bu klasör, “hello world” modelini **masaüstünde eğitip .tflite dosyası üretmek** için kullanılır.

Ama bu **LoPy4 içinde doğrudan çalışmaz**, çünkü LoPy4 yalnızca **C/C++ kodu + TFLite Micro runtime** çalıştırabilir.

Yani özetle:

- Bu gördüğün dosyalar (evaluate.py, train.py) eğitimi masaüstünde yapıyor.
 - LoPy4 üzerinde çalıştırmak için o eğitim sonucunda üretilen .tflite modelini alıp, **C++ içine gömmemiz (byte dizisine çevirmemiz)** gerekiyor.
 - Sonra onu Pycom firmware’ine dahil edip, MicroInterpreter ile tahmin yaptırıyoruz.
-

 **Şimdi iki seçeneğin var:**

◆ **Seçenek 1 — “Model eğitimi” ve ardından cihazda inference**

1. Masaüstünde (PC’de) train.py’yi çalıştır:
2. `cd ~/lopy4/tflite-micro/tensorflow/lite/micro/examples/hello_world/`
3. `python3 train.py`

Bu adım sonunda genelde models/ klasöründe bir dosya oluşur:

models/sine_model.tflite

4. Bu .tflite dosyasını C dizisine dönüştür:
5. `xxd -i models/sine_model.tflite > model_data.cc`

(Bu, unsigned char g_model[] = {...} şeklinde C array üretir.)

6. Bu model_data.cc dosyasını LoPy4 firmware’ine (pycom-micropython-sigfox/esp32/tflm/) kopyala.
 7. Son olarak main_functions.cc ve constants.cc (örnek TFLM kodları) kullanılarak **LoPy4 üzerinde** bu modeli çalıştırabiliriz.
-

◆ **Seçenek 2 — Ben senin için hazır bir “hello_world” C++ sürümü vereyim**

Eğer istersen hiç model eğitimiyle uğraşmadan,
ben sana doğrudan:

- Model verisini (g_model_data[])
- Inference kodunu (tflm_hello_module.c) hazır verebilirim.

Sen sadece bunları esp32/tflm/ içine atıp firmware’i yeniden build edeceksin.

Hangisini istersin?

- 1 — Kendi modelini train.py ile eğitip dönüştürmek
- 2 — Hazır “hello world” (sin(x) tahmin eden) C++ kodunu doğrudan almak
 - “hello world” (sin(x) tahmin eden) C++ kodunu doğrudan almak bunu yapalım zaten sana attığım repoda olmalı

Evet, doğru 👍

Senin attığın repo (tensorflow/tflite-micro) içinde “hello world” örneği zaten var.

Bu örnek, **siny(x)** fonksiyonunu tahmin eden küçük bir sinir ağını içeriyor ve tam olarak TFLite Micro’nun temel demosu.

Aşağıda bu örneği nerede bulacağını ve nasıl çalıştıracağını adım adım anlatayım 📝

📁 1. Örnek kodun yeri

Repo içinde örnek kodun yolu:

tensorflow/lite/micro/examples/hello_world/

Bu klasörün içinde şu dosyaları bulacaksın:

main.cc

main_functions.cc

main_functions.h

constants.cc

constants.h

model_data.cc

model_data.h

output_handler.cc

output_handler.h

2. Bu örneğin yaptığı şey

- `model_data.cc`: $\sin(x)$ fonksiyonunu yaklaşık olarak öğrenmiş küçük bir NN modelini (.tflite) binary formatta içerir.
 - `main_functions.cc`: inference döngüsünü çalıştırır (modeli yükler, tensorleri ayarlar, girişleri değiştirip sonuçları alır).
 - `output_handler.cc`: sonucu basitçe bir “print” veya “LED blink” olarak gösterir (MCU’ya göre değişebilir).
-

3. Sadece C++ tarafını derlemek istersen

Eğer Pycom LoPy4’e doğrudan MicroPython üzerinden değil, C++ firmware ile çalıştıracaksan:

Bu klasör kendi başına TFLite Micro ile derlenebilir.

Minimal örnek (masaüstü test için) şöyle yapılır:

```
cd tensorflow/lite/micro/tools/make
```

```
make -f Makefile TARGET=linux hello_world
```

Bu komut:

- “hello world” örneğini Linux hedefi için derler,
 - `gen/linux_x86_64_default/bin/hello_world` dosyasını üretir,
 - çalıştırınca terminalde $\sin(x)$ çıktıları görürsün.
-

4. LoPy4 (ESP32 tabanlı) için uyarılama

LoPy4 bir **ESP32** tabanlı cihazdır, yani bu örneği doğrudan derleyemezsin.

Bunun için:

1. `hello_world` klasörünü bağımsız bir ESP-IDF projesine taşıman gerekir.
2. `model_data.cc`, `main_functions.cc`, `constants.cc` dosyalarını dahil edip `setup()` ve `loop()` benzeri yapıya yerleştirirsin.
3. `output_handler.cc` içinde `printf` yerine LED blink veya UART print kullanırsın.

Bunu birazdan birlikte adım adım yapabiliriz.

- Bunu yaparken `make` komutunda hata aldık:
bilmuh@lab3pc22:~/lopy4/tflite-micro/tensorflow/lite/micro/tools/make\$
`make -f Makefile TARGET=linux hello_world` find:

'tensorflow/lite/micro/examples/': Böyle bir dosya ya da dizin yok find:
'tensorflow': Böyle bir dosya ya da dizin yok find: 'tensorflow': Böyle bir dosya ya da dizin yok make:
tensorflow/lite/micro/tools/make/flatbuffers_download.sh: Böyle bir dosya ya da dizin yok Makefile:633: *** Something went wrong with the flatbuffers download: . Durdu.

Sorunun sebebi aslında **senin şu anda tools/make klasörünün içinden komutu çalıştırmaman**, ama Makefile'in **repo kök dizininden** çağrılmasını beklemesi.

Ayrıca flatbuffers gibi bağımlılıkları indirmesi gerekiyor ama bunu da bulamıyor çünkü yol hatalı.

Sorunun Özeti

Hata:

find: 'tensorflow/lite/micro/examples/': Böyle bir dosya ya da dizin yok
make: tensorflow/lite/micro/tools/make/flatbuffers_download.sh: Böyle bir dosya ya da dizin yok

Bu demek ki make komutunu yanlış dizinde çalıştırdık.

Makefile içindeki yollar **repo kökünden görelili** tanımlı, o yüzden make komutu **root klasörden** verilmeli.

Çözüm (Linux için)

Aşağıdaki adımları sırayla uygula:

```
# repo kök dizinine dön  
cd ~/lopy4/tflite-micro
```

```
# make komutunu tam buradan ver
```

```
make -f tensorflow/lite/micro/tools/make/Makefile TARGET=linux  
hello_world
```

Bu komut doğru yolları bulacak ve:

- flatbuffers kütüphanesini otomatik indirip kuracak (biraz zaman alabilir),
- hello_world örneğini derleyecek,
- çıktığı şu klasöre koyacak:

```
tensorflow/lite/micro/tools/make/gen/linux_x86_64_default/bin/hello_world
```

Test Etme

Derleme bitince şu komutla test et:

```
./tensorflow/lite/micro/tools/make/gen/linux_x86_64_default/bin/hello_w  
orld
```

Eğer her şey yolundaysa terminalde aşağıdakine benzer bir çıktı görürsün



```
x_value: 0.00, y_pred: 0.00
```

```
x_value: 0.10, y_pred: 0.10
```

```
x_value: 0.20, y_pred: 0.19
```

```
...
```

- Derlemeler tamamlandı lopy4 le çalıştırmaya geçtik:

1 MicroPython firmware klasörüne dosyaları ekle

Bizim Linux derlemesinden farklı olarak, LoPy4 üzerinde C/C++ dosyalarını **Pycom firmware** içinde derleyeceğiz.

Terminalde şunları yap:

```
cd ~/lopy4/pycom-micropython-sigfox/esp32/tflm/
```

Buraya kopyalıyoruz:

```
# Model binary ve fonksiyonlar
```

```
cp ~/lopy4/tflite-micro/tensorflow/lite/micro/examples/hello_world/*.cc .
```

```
cp ~/lopy4/tensorflow/lite/micro/examples/hello_world/*.h .
```

Eğer .h dosyaları yoksa sorun değil, sadece .cc dosyalarını kopyala.

2 tflm_hello_module.c oluştur

MicroPython'dan çağırabileceğimiz bir modül oluşturacağız. Örnek minimal içeriği:

```
// tflm_hello_module.c
```

```
#include "py/obj.h"
```

```
#include "py/runtime.h"
```

```
extern void setup_hello_world(); // main_functions.cc içindeki setup  
fonksiyonu
```

```
extern void loop_hello_world(); // main_functions.cc içindeki loop fonksiyonu
```

```

STATIC mp_obj_t hello_run(void) {
    setup_hello_world();
    loop_hello_world();
    return mp_const_none;
}

STATIC MP_DEFINE_CONST_FUN_OBJ_0(hello_run_obj, hello_run);

STATIC const mp_rom_map_elem_t hello_module_globals_table[] = {
    { MP_ROM_QSTR(MP_QSTR_run), MP_ROM_PTR(&hello_run_obj) },
};

STATIC MP_DEFINE_CONST_DICT(hello_module_globals,
    hello_module_globals_table);

const mp_obj_module_t hello_module = {
    .base = { &mp_type_module },
    .globals = (mp_obj_dict_t*)&hello_module_globals,
};

// Bu modül build sırasında MicroPython'a eklenmeli
MP_REGISTER_MODULE(MP_QSTR_hello, hello_module);

```

- Bu modül MicroPython'dan şu şekilde çalıştırılabilir:

```

import hello

hello.run()

```

3 Firmware'i derle ve LoPy4'e yükle

Pycom'un ESP32 build sistemi ile derle:

```
cd ~/lopy4/pycom-micropython-sigfox/esp32
```

```
make BOARD=LOPY4 deploy
```

- Cihazın bağlı olduğundan emin ol.
- deploy komutu firmware'i LoPy4'e yükler.

MicroPython'da test et

Cihaz açıldıktan sonra **REPL** üzerinden:

```
import hello
```

```
hello.run()
```

- Terminalde veya cihazın LED/seri çıkışında **hello_world sinüs tahmin çıktıları** görünmeli.

Özet Akış:

1. LoPy4 bağlı 
2. .cc ve .h dosyalarını firmware içine kopyala 
3. tflm_hello_module.c oluştur 
4. Firmware'i derle ve deploy et 
5. MicroPython'dan hello.run() çağır 

- Buradan da olmayınca ESP-IDF yöntemi önerdi fakat lopy4 kendi özelliklerini kaybetmesin diye yapmadık.

Tam olarak öyle: **ESP-IDF ile derlersen LoPy4'ün mevcut**

MicroPython/Pycom firmware'inin üstüne yazarsın, yani WiFi, LoRa, Bluetooth vs. gibi Pycom özellikleri gider. Bu yöntem sadece "tam C/C++ TFLite Micro firmware" çalıştırmak için uygundur; Pycom özelliklerini kaybedersin.

LoPy4'ün özelliklerini kaybetmeden TFLite Micro çalıştırmak

Bunu yapmak için **MicroPython üstünde TFLite Micro'yu port etmen gerekir**. Yani Pycom'un firmware'ini silmeden:

1. **MicroPython firmware kullanmaya devam edersin** → WiFi, LoRa, Bluetooth vs. aktif kalır.

2. TFLite Micro **MicroPython modülü olarak veya C extension olarak firmware içine eklenir.**
3. Modeli çalıştırmak için MicroPython'dan çağırırsın, örnek:

```
import tflite_micro  
  
result = tflite_micro.run_model(input_data)  
  
print(result)
```

Mevcut seçenekler

1. **C/C++ ESP-IDF yöntemi** → LoPy4 tamamen TFLite Micro firmware olur, Pycom özellikleri kaybolur.
2. **MicroPython üstüne portlama (güvenli yöntem)** → WiFi, LoRa vs. çalışır, ancak:
 - Derleme ve portlama daha karmaşık.
 - TFLite Micro'yu MicroPython'a entegre etmek gerekiyor (C modül yazmak veya halihazırdaki MicroPython TFLite Micro portlarından faydalanmak).

- Son konuşma:

LoPy4 üzerinde TensorFlow Lite Micro (TinyML) örneği *ESP-IDF kullanılmadan* çalıştırılmaz.

Yani MicroPython içine .tflite modeli koyup direkt çalıştıramazsın.

Bunun nedenini *çok net ve anlaşılır şekilde* adım adım açıklıyorum:

■ NEDEN LoPy4 ÜZERİNDE TFLITE MICRO'YU MICROPYTHON İLE DOĞRUDAN ÇALIŞTIRAMAZSIN?

✓ 1. TFLite Micro C/C++ ile derlenir (Python ile değil)

TensorFlow Lite Micro:

- Saf **C/C++ kütüphanesidir**
- Model çalıştırma kodu (micro_interpreter.cc, kernels, ops) tamamen C++'tır
- MicroPython'ın içinden doğrudan C++ çağırmak **mümkün değildir**

👉 MicroPython yalnızca **C ile yazılmış modülleri** load eder.

✓ 2. TFLite Micro ESP32 üzerinde çalışması için ESP-IDF gerekir

LoPy4 bir **ESP32 tabanlı cihazdır**.

ESP32’de TFLite Micro çalıştırmak için gerekli olanlar:

- FreeRTOS thread’leri
- Tensörler için statik memory arena
- CMSIS/ESP32 optimized ops
- ESP-IDF build sistemi
- linker script ayarları
- tensör arena’nın RAM/IRAM’e yerleştirilmesi

💡 Bunlar *MicroPython firmware’inde yoktur*.

MicroPython sadece basit C modüllerini kabul eder.
Ama TFLite Micro **binlerce satır C++ kodu** ile gelir.

✓ 3. Hello World örneği bile C++ derlemesi ister

Örnekteki dosyalar:

main_functions.cc

hello_world_model_data.cc

constants.cc

Ve bunlar şunlara bağımlı:

micro_interpreter.cc

all_ops_resolver.cc

kernel_xxx.cc

MicroPython bu *.cc dosyalarını derleyemez.

✓ 4. Pycom LoPy4 firmware’i ESP-IDF üzerine kurulur

Pycom firmware şöyle yapılandırılır:

+ MicroPython core

+ Pycom API

- ESP-IDF alt katmanı minimum seviyede

Ama TFLite Micro, ESP-IDF'in **tam HAL** (Hardware Abstraction Layer) katmanını ister.

Pycom bunu expose etmez (erişemezsin).

Yani Pycom firmware içinde ESP-IDF full seviyede yok → TFLite Micro build edilemez.

■ Peki NEDEN ESP-IDF KULLANMAK ZORUNDAYIZ?

Çünkü TFLite Micro:

- Tensör arena için IRAM/DRAM mapping ister
- Memory alignment ister
- Control flow scheduler kullanır
- Hardware optimized convolution kernel çalıştırır
- Interrupt-safe çağrılar yapar

Bunların hepsi **ESP-IDF tarafından sağlanır**.

MicroPython → bunları SAĞLAMAZ.

■ O ZAMAN LO PY4'TE TINYMML NASIL ÇALIŞTIRILIR?

SEÇENEK 1 — ESP-IDF ile bağımsız firmware derlemek (Tek gerçek çözüm) ✓

Bu yöntemde:

1. Pycom firmware'i silersin
2. ESP-IDF yüklersin
3. TFLite Micro örneklerini ESP32 için build edersin
4. Cihazda direkt C++ olarak çalıştırırsın

Bu %100 çalışan yöntemdir.

SEÇENEK 2 — *MicroPython içinde TFLite Micro kullanmak İMKANSIZ* ✗

Çünkü:

- port yok
- wrapper yok

- binary boyutu MicroPython'a sığmıyor
- C++ zorunlu
- model çalıştırma için düşük seviye erişim gerekiyor

■ EN UYGUN ÇÖZÜM NE?

👉 Eğer illa **LoPy4** üzerinde TinyML çalıştırmak istiyorsan:

● En mantıklısı: **ESP-IDF + TFLite Micro ile özel firmware üretmek.**

Ben istersen:

✓ **ESP-IDF kurulumunu**

✓ **LoPy4'e uygun TFLite Micro projesi oluşturmayı**

✓ **Hello World örneğini LoPy4 için build etmeyi**

✓ **Flash etmeyi**

adım adım tam rehber olarak çıkarayım.

? İstersen şimdi LoPy4 için çalışan bir ESP-IDF + TFLite Micro projesi oluşturmaya başlayalım mı?

"evet" dersin direkt başlıyoruz.